Appendix — ME 2030 Loop Project

Course: ME 2030 Spring 2026 **Project:** Loop-the-Loop Drop Height Prediction **Competition Parameters:** $R_{loop} = 4.75$ in, $\theta = 56^\circ$

Table of Contents

Section	Title	Page Reference
A.1	Reference Case Derivations	
A.1.1	Frictionless Sliding Block ($h = \frac{5}{2}R$)	
A.1.2	Sphere Rolling Without Slipping on Flat Surface ($h = \frac{27}{10}R$)	
A.2	Two-Rail Contact Geometry (Bachman 1985)	
A.3	Full Energy Balance with Losses (Our Model)	
A.4	Numerical Comparison at $R_{loop} = 5$ in, $\theta = 20^\circ/50^\circ$	
A.5	Slip Regime Correction (Post-Competition Analysis)	
A.6	Source Code Listings	
A.6.1	README — How the Code Was Developed	
A.6.2	`physics/__init__.py`	
A.6.3	`physics/constants.py`	
A.6.4	`physics/geometry.py`	
A.6.5	`physics/energy.py`	
A.6.6	`physics/ode_model.py`	
A.6.7	`app.py` (Streamlit UI)	
A.7	References	

All equations use notation from the ME 2030 Dynamics Vocabulary sheet and follow Hibbeler 15th Ed. conventions. Rolling-body kinematics are derived using the **instant center of zero velocity (IC)** method.

A.1 Reference Case Derivations

A.1.1 Frictionless Sliding Block (Reference Case 1)

Setup: Block of mass m released from rest at height h above the bottom of a vertical circular loop of radius R. No friction, no rotation.

Step 1 — Apply Work-Energy Theorem from release to top of loop. From the equation sheet:

$$T_1 + V_1 = T_2 + V_2$$

With $T_1 = 0$ (released from rest), $V_1 = mgh$, $V_2 = mg(2R)$, $T_2 = \frac{1}{2}mv_{top}^2$:

$$mgh = mg(2R) + \tfrac{1}{2}mv_{top}^2$$

Step 2 — Apply Newton's 2nd Law (radial) at top of loop. From the equation sheet, $a_n = v^2/\rho$. At the top, both weight and normal force point toward the loop center:

$$\sum F_n = ma_n \quad \Rightarrow \quad N + mg = \frac{mv_{top}^2}{R}$$

Step 3 — Apply critical condition $N = 0$:

$$v_{top}^2 = gR$$

Step 4 — Substitute (A.2) into (A.1) and solve:

$$mgh = 2mgR + \tfrac{1}{2}m(gR) \quad \Rightarrow \quad h = 2R + \tfrac{1}{2}R$$

$$\boxed{h_{min} = \tfrac{5}{2}R}$$

For $R = 5$ in: $h_{min} = 12.5$ in.

A.1.2 Sphere Rolling Without Slipping on Flat Surface (Reference Case 2)

Setup: Solid sphere of mass m, radius r rolling without slipping. Loop radius R.

Step 1 — Identify the instant center (IC). For rolling without slipping, the contact point has zero velocity, defining the instant center of zero velocity. The sphere center G is at distance r from the IC.

Step 2 — Rolling constraint from IC kinematics. From the equation sheet, for a body with a fixed point: $\mathbf{v}_A = \boldsymbol{\omega} \times \mathbf{r}_A$. Applied with the IC as the fixed point:

$$v_G = \omega r$$

Step 3 — Kinetic energy using IC form. From the equation sheet:

$$T = \tfrac{1}{2}I_{IC}\omega^2$$

By the parallel axis theorem (equation sheet: $I_A = I_G + md^2$), with $I_G = \frac{2}{5}mr^2$ for a solid sphere:

$$I_{IC} = I_G + mr^2 = \tfrac{2}{5}mr^2 + mr^2 = \tfrac{7}{5}mr^2$$

$$T = \tfrac{1}{2}\left(\tfrac{7}{5}mr^2\right)\omega^2 = \tfrac{7}{10}m(\omega r)^2 = \tfrac{7}{10}mv^2$$

Step 4 — Energy conservation:

$$mgh = mg(2R) + \tfrac{7}{10}mv_{top}^2$$

Step 5 — Critical condition (radial force balance unchanged by rotation):

$$v_{top}^2 = gR$$

Step 6 — Substitute (A.5) into (A.4) and solve:

$$mgh = 2mgR + \tfrac{7}{10}m(gR) \quad \Rightarrow \quad h = 2R + \tfrac{7}{10}R$$

$$\boxed{h_{min} = \tfrac{27}{10}R}$$

For $R = 5$ in: $h_{min} = 13.5$ in.

A.2 Two-Rail Contact Geometry (Bachman 1985)

Setup: Solid sphere of radius R rests on two parallel cylindrical rails, each of radius r_{rail}, with center-to-center spacing s. For our track: $r_{rail} = 1.5$ mm, $s = 14$ mm (computed as outside-to-outside 17 mm minus one rail diameter 3 mm).

Step 1 — Cross-section geometry. Looking along the track, rail centers lie at $\pm s/2$ from the symmetry axis. The sphere center sits at height h_{offset} above the rail-center plane. Since sphere–rail contact is external, the center-to-center distance is $R + r_{rail}$.

Step 2 — Apply Pythagorean theorem. The right triangle formed by the sphere center, one rail center, and the midpoint between rails gives:

$$(R + r_{rail})^2 = (s/2)^2 + h_{offset}^2$$

$$\boxed{h_{offset} = \sqrt{(R + r_{rail})^2 - (s/2)^2}}$$

The contact half-angle α (from vertical to the line from sphere center to contact point):

$$\sin\alpha = \frac{s/2}{R + r_{rail}}, \qquad \cos\alpha = \frac{h_{offset}}{R + r_{rail}}$$

Step 3 — Identify the IC axis. For rolling without slipping at *both* rails, the velocity at each contact point is zero. The line connecting the two contact points is therefore the IC axis (parallel to the rails, perpendicular to direction of travel). The sphere center G is at perpendicular distance r_{eff} from this axis:

$$\boxed{r_{eff} = R\cos\alpha = \frac{R \cdot h_{offset}}{R + r_{rail}}}$$

Step 4 — Rolling constraint from IC kinematics. Using $\mathbf{v}_A = \boldsymbol{\omega} \times \mathbf{r}_A$ with the IC as the fixed point:

$$v_G = \omega \cdot r_{eff} \quad \Rightarrow \quad \omega = \frac{v}{r_{eff}}$$

Since $r_{eff} < R$, the sphere spins **faster** than it would on a flat surface at the same translational speed.

Step 5 — Kinetic energy using IC form. From the equation sheet, $T = \frac{1}{2} I_{IC} \omega^2$. By parallel axis theorem:

$$I_{IC} = I_G + m r_{eff}^2 = \tfrac{2}{5} m R^2 + m r_{eff}^2$$

Substituting $\omega = v / r_{eff}$:

$$T = \tfrac{1}{2} \left[\tfrac{2}{5} m R^2 + m r_{eff}^2 \right] \left(\frac{v}{r_{eff}} \right)^2$$

$$\boxed{T = \tfrac{1}{2} m v^2 \left[1 + \tfrac{2}{5} \left(\frac{R}{r_{eff}} \right)^2 \right] = \tfrac{1}{2} m v^2 \cdot KE_{factor}}$$

Numerical values for our balls:

Ball	R **(mm)**	h_{offset} **(mm)**	r_{eff} **(mm)**	R/r_{eff}	KE_{factor}
Steel	7.00	4.82	3.97	1.762	2.241
Plastic	7.50	5.66	4.71	1.591	2.013
Rubber	7.95	6.35	5.34	1.490	1.888

For comparison, a flat surface gives $KE_{factor} = 1.4$. The two-rail correction adds **35–60%** more rotational energy.

A.3 Full Energy Balance with Losses (Our Model)

Setup: Sphere of mass m on two rails, released from height $h_{release}$ above the bottom of the loop. Loop has rail-centerline radius R_{loop}. Ramp at angle θ. Center-of-mass path radius in the loop: $R_c = R_{loop} - h_{offset}$.

Step 1 — Identify all energy terms.

Ball center heights (relative to bottom of loop track surface): - Release: $y_{cm,1} = h_{release} + h_{offset} \cos\theta$ - Loop top: $y_{cm,2} = 2R_{loop} - h_{offset}$

Kinetic energies: - Release: $T_1 = 0$ (from rest) - Loop top: $T_2 = \frac{1}{2} m v_{top}^2 \cdot KE_{factor}$

Non-conservative work (each from $U = \int \mathbf{F} \cdot d\mathbf{r}$ on the equation sheet): - Ramp rolling resistance: $W_{ramp} = C_{rr}\, mg \cos\theta \cdot L_{ramp} = C_{rr}\, mg\, h_{release} / \tan\theta$ - Loop rolling resistance: $W_{loop} = C_{rr} \cdot N_{avg} \cdot 2\pi R_c \approx C_{rr} \cdot 3mg \cdot 2\pi R_c$ - Transition jerk loss: $W_{trans} = f_{trans} \cdot T_{entry} \approx f_{trans}\, mg\, [h_{release} - h_{offset}(1 - \cos\theta)]$

Step 2 — Apply Work-Energy Theorem with non-conservative work (U_{NC} on equation sheet):

$$T_1 + V_1 = T_2 + V_2 + U_{NC}$$

$$mg\, y_{cm,1} = mg\, y_{cm,2} + \tfrac{1}{2} m v_{top}^2\, KE_{factor} + W_{ramp} + W_{loop} + W_{trans}$$

Step 3 — Apply critical condition (Newton's 2nd Law, radial, $N = 0$ at top):

At the loop top, both gravity and normal force point toward the loop center:

$$N + mg = \frac{m v_{top}^2}{R_c} \quad \Rightarrow \quad \boxed{v_{top}^2 = g R_c}$$

Step 4 — Substitute and expand:

$$mg\,[h_{release} + h_{offset}\cos\theta] = mg[2R_{loop} - h_{offset}] + \tfrac{1}{2} m (g R_c) KE_{factor} + W_{ramp} + W_{loop} + W_{trans}$$

Step 5 — Collect $h_{release}$ terms (it appears on both sides through W_{ramp} and W_{trans}):

$$\begin{aligned} mg\, h_{release} \left[1 - \frac{C_{rr}}{\tan\theta} - f_{trans}\right] \\ = mg\,[2R_{loop} - h_{offset}(1+\cos\theta)] \\ + \tfrac{1}{2} m (g R_c) KE_{factor} + W_{loop} \\ - f_{trans}\, mg\, h_{offset}(1-\cos\theta) \end{aligned}$$

Step 6 — Solve algebraically:

$$\boxed{h_{release} = \frac{2R_{loop} - h_{offset}(1+\cos\theta) + \frac{R_c}{2} KE_{factor} + 6\pi C_{rr} R_c - f_{trans}\, h_{offset}(1-\cos\theta)}{1 - \frac{C_{rr}}{\tan\theta} - f_{trans}}}$$

Implementation: `physics/energy.py:two_rail_height` (lines 47–125).

A.4 Numerical Comparison at $R_{loop} = 5$ in, $\theta = 20^\circ$ and 50°

Per project Section II requirement.

Model	$\theta = 20^\circ$ **(in)**	$\theta = 50^\circ$ **(in)**	Δ **vs Block (20°/50°)**	Δ **vs Rolling (20°/50°)**
Frictionless block	12.50	12.50	—	—
Rolling on flat	13.50	13.50	—	—
Steel (two-rail + losses)	16.1	16.1	+3.6 / +3.6	+2.6 / +2.6
Plastic (two-rail + losses)	17.3	16.8	+4.8 / +4.3	+3.8 / +3.3
Rubber (two-rail + losses)	29.0	24.0	+16.5 / +11.5	+15.5 / +10.5

Drivers of the differences: - The two-rail r_{eff} correction alone adds 1.5–2 in for steel and plastic regardless of angle (raises rotational-energy fraction from 28.6% on flat to ~55% of total KE). - Rolling resistance dominates the rubber predictions ($C_{rr} \approx 0.08$). - Steeper angles reduce required height because ramp-friction work scales as $1/\tan\theta$. Reference cases show no angle dependence.

A.5 Slip Regime Correction (Post-Competition Analysis)

Motivation: At the competition $\theta = 56^\circ$, our model underpredicted heights for steel and plastic by 12–15 inches. The dominant cause is violation of the rolling-without-slipping assumption.

Step 1 — Newton's 2nd Law along the ramp. From equation sheet: $\mathbf{F} = m\mathbf{a}_G$. Tangential component:

$$mg\sin\theta - f_s = ma_G$$

Step 2 — Moment equation about the IC. From equation sheet: $\sum M_{IC} = I_{IC}\alpha$. Both friction (at contact) and normal force pass through the IC, contributing no moment. Only gravity at G (perpendicular distance r_{eff}) contributes:

$$mg\sin\theta \cdot r_{eff} = I_{IC}\alpha$$

With $I_{IC} = \frac{2}{5}mR^2 + mr_{eff}^2$ and $a_G = \alpha r_{eff}$:

$$a_G = \frac{g\sin\theta}{1 + \frac{2}{5}(R/r_{eff})^2} = \frac{g\sin\theta}{KE_{factor}}$$

Step 3 — Solve for required static friction.

$$f_s = mg\sin\theta - ma_G = mg\sin\theta\left(\frac{KE_{factor} - 1}{KE_{factor}}\right)$$

Step 4 — Apply no-slip condition $f_s \leq \mu_s N$ where $N = mg\cos\theta$:

$$\boxed{\mu_s \geq \tan\theta \cdot \frac{KE_{factor} - 1}{KE_{factor}} \quad \Leftrightarrow \quad \theta_{crit} = \arctan\left(\mu_s \cdot \frac{KE_{factor}}{KE_{factor} - 1}\right)}$$

For comparison, a flat-surface rolling sphere has $KE_{factor} = 7/5$, giving the textbook $\mu_s \geq \frac{2}{7}\tan\theta$. Our two-rail geometry has a **higher** KE_{factor}, which **lowers** the critical angle.

Step 5 — Numerical critical angles per ball. Friction test: $\mu_s \approx \tan(12^\circ) = 0.213$.

Ball	KE_{factor}	θ_{crit} **at** $\mu_s = 0.213$
Steel	2.241	**21.0°**
Plastic	2.013	**22.9°**
Rubber (if $\mu_s = 0.213$)	1.888	**24.4°**

For rubber not to slip at $\theta = 56^\circ$, μ_{rubber} must satisfy $\mu_{rubber} \geq 0.697$ — plausible for rubber-on-steel.

Conclusion: At competition $\theta = 56^\circ$, steel and plastic exceeded their critical angles by more than a factor of 2 and SLIPPED on the ramp.

Step 6 — Slip energy loss. Once slipping, kinetic friction does work along the ramp ($U = \int \mathbf{F} \cdot d\mathbf{r}$):

$$\boxed{W_{slip} = \mu_k \, mg \cos\theta \cdot L_{ramp} = \mu_k \, mg \cdot \frac{h_{release}}{\tan\theta}}$$

This term replaces W_{ramp} in the energy balance (§A.3 Step 1) when slip occurs. Re-running the model with this correction at $\theta = 56^\circ$ yields predictions of approximately 28–32 in for steel and plastic, matching the experimental results.

A.6 Source Code Listings

The following Python source files implement the model. Run with `streamlit run app.py` after `pip install -r requirements.txt`.

A.6.1 README — How the Code Was Developed

The model was built from first principles using the equations on the ME 2030 Dynamics Vocabulary sheet (Hibbeler 15th Ed. conventions). Each formula was independently verified against published literature (Bachman 1985, Wang et al. 2021, Bertran 2020) before implementation. AI assistance (Claude) was used to generate code structure; all physics and equations were hand-derived, audited via four independent code reviews, and validated with a full ODE simulation that matches the analytical solver to within 1% in the no-friction limit.

Dynamics principles utilized: 1. Work-Energy Theorem ($T_1 + V_1 + U_{NC} = T_2 + V_2$) 2. Kinetic Energy of a Rigid Body ($T = \frac{1}{2}mv_G^2 + \frac{1}{2}I_G\omega^2$, equivalently $\frac{1}{2}I_{IC}\omega^2$) 3. Newton's 2nd Law (linear and rotational forms) 4. Mass Moment of Inertia + Parallel Axis Theorem 5. Instant Center of Zero Velocity (IC) method for rolling-body kinematics 6. Normal-Tangential Acceleration ($a_n = v^2/\rho$) 7. Rolling Constraint via IC: $v_G = \omega \cdot r_{eff}$

File structure: - `physics/constants.py` — Ball masses, radii, rail geometry, unit conversions - `physics/geometry.py` — Two-rail contact geometry (Bachman 1985) - `physics/energy.py` — Analytical energy-balance solver + waterfall breakdown - `physics/ode_model.py` — scipy `solve_ivp` simulation with terminal events - `app.py` — Streamlit UI dashboard

Run instructions:

```
pip install -r requirements.txt
streamlit run app.py
```

The app provides interactive sliders for loop radius, ramp angle, ball type, and uncertain parameters. The waterfall section breaks down how each physics correction contributes to the final prediction.

A.6.2 `physics/__init__.py` (6 lines)

```
"""Physics engine for the loop-the-loop dynamics model."""
from .constants import BALLS, G, RAIL_RADIUS, RAIL_SPACING, BallProperties, INCHES_TO_METERS,
        METERS_TO_INCHES
from .geometry import compute_contact_geometry, compute_effective_loop_radius
```

```
from .energy import (frictionless_block_height, flat_rolling_height,
                     two_rail_height, compute_all_predictions, compute_waterfall)
from .ode_model import simulate
```

A.6.3 `physics/constants.py` (66 lines)

```
"""
Known constants and ball/track data for the loop-the-loop project.
All values from the ME 2030 project description unless noted otherwise.

Dynamics principles: This file contains the given physical parameters
for the three test spheres and the track geometry. Heights are measured
relative to the bottom of the loop (the competition datum).
"""
from dataclasses import dataclass

G = 9.81  # m/s^2, gravitational acceleration

# Track geometry (given in project description)
# DATUM: All heights measured from the track surface at the bottom of the loop.
# The project says "drop heights should be given relative to the bottom of the loop."
RAIL_DIAMETER_MM = 3.0
RAIL_RADIUS_MM = RAIL_DIAMETER_MM / 2          # 1.5 mm
RAIL_OUTSIDE_SPACING_MM = 17.0
# Center-to-center = outside-to-outside minus one diameter:
# 17mm outside-to-outside - 3mm rail diameter = 14mm center-to-center
RAIL_CENTER_SPACING_MM = RAIL_OUTSIDE_SPACING_MM - RAIL_DIAMETER_MM  # 14.0 mm

# Convert to meters for calculations
RAIL_RADIUS = RAIL_RADIUS_MM / 1000
RAIL_SPACING = RAIL_CENTER_SPACING_MM / 1000    # 0.014 m center-to-center

# Unit conversions
INCHES_TO_METERS = 0.0254
METERS_TO_INCHES = 1 / INCHES_TO_METERS

@dataclass
class BallProperties:
    """Physical and modeling properties for one of the three test spheres."""
    name: str
    mass_kg: float
    radius_m: float
    mu_sliding: float          # sliding friction coefficient
    C_rr: float                # rolling resistance coefficient (tunable default)
    color: str                 # for plotting

BALLS = {
    "Steel": BallProperties(
        name="Stainless Steel",
        mass_kg=0.011,
        radius_m=0.007,
        mu_sliding=0.213,       # tan(12 deg), from friction test
        C_rr=0.002,             # default, tunable
        color="#4A90D9",
    ),
    "Plastic": BallProperties(
        name="Plastic",
        mass_kg=0.003,
        radius_m=0.0075,
        mu_sliding=0.213,       # tan(12 deg), from friction test
```

```
        C_rr=0.015,             # default, tunable
        color="#E8A838",
    ),
    "Rubber": BallProperties(
        name="Nitrile Rubber",
        mass_kg=0.0028,
        radius_m=0.00795,
        mu_sliding=0.35,        # estimated (tipped before slipping, so > 0.213)
        C_rr=0.08,              # default, tunable (significant due to hysteresis)
        color="#D94A4A",
    ),
}
```

A.6.4 `physics/geometry.py` (81 lines)

```
"""
Two-rail contact geometry for a sphere rolling on parallel cylindrical rails.

References:
  - Bachman (1985), "Sphere rolling down a grooved track," Am. J. Phys. 53(8), 765
  - West, Barrett & Vender (2024), "Deep in Galileo's Groove," The Physics Teacher 62(2), 104

Key insight: the effective rolling radius r_eff < R_ball because the contact
points are elevated above the lowest point of the sphere. The ball spins faster
for a given translational speed, storing more energy in rotation.

The instantaneous axis of rotation passes through the two contact points,
not through the bottom of the sphere. The lever arm for converting angular
velocity to linear velocity is r_eff, not R.
"""
import numpy as np
from .constants import RAIL_RADIUS, RAIL_SPACING


def compute_contact_geometry(R_ball: float, r_rail: float = RAIL_RADIUS,
                             s: float = RAIL_SPACING) -> dict:
    """
    Compute all derived geometric quantities for a sphere on two cylindrical rails.

    Parameters:
        R_ball: ball radius (m)
        r_rail: rail radius (m), default from constants
        s: rail center-to-center spacing (m), default from constants

    Returns dict with:
        alpha: contact half-angle (rad)
        h_offset: height of ball center above rail center plane (m)
        r_eff: effective rolling radius (m)
        beta: rotational KE multiplier = (2/5)*(R/r_eff)^2
        KE_factor: total KE factor = 1 + beta
    """
    half_s = s / 2
    R_plus_r = R_ball + r_rail

    # Verify the ball is large enough to sit on the rails
    if R_plus_r <= half_s:
        raise ValueError(
            f"Ball radius {R_ball*1000:.1f}mm + rail radius {r_rail*1000:.1f}mm "
            f"= {R_plus_r*1000:.1f}mm must exceed half-spacing {half_s*1000:.1f}mm"
        )
```

```python
    # Contact half-angle: angle from vertical to line from ball center to rail center
    alpha = np.arcsin(half_s / R_plus_r)

    # Height of ball center above the plane containing rail centers
    h_offset = np.sqrt(R_plus_r**2 - half_s**2)

    # Effective rolling radius: perpendicular distance from rotation axis to CM velocity
    # r_eff = R * cos(alpha) = R * h_offset / (R + r_rail)
    r_eff = R_ball * h_offset / R_plus_r

    # Rotational KE multiplier: (2/5)*(R/r_eff)^2
    # For a solid sphere I = 2/5 mR^2, and omega = v/r_eff, so
    # KE_rot = (1/2)(2/5)mR^2 * (v/r_eff)^2 = (1/5)m(R/r_eff)^2 * v^2
    beta = (2.0 / 5.0) * (R_ball / r_eff) ** 2

    # Total KE = (1/2)mv^2 * KE_factor where KE_factor = 1 + beta
    KE_factor = 1.0 + beta

    return {
        "alpha": alpha,
        "h_offset": h_offset,
        "r_eff": r_eff,
        "beta": beta,
        "KE_factor": KE_factor,
    }


def compute_effective_loop_radius(R_loop: float, h_offset: float) -> float:
    """
    CM path radius inside the loop.

    The ball center is offset inward from the rail surface by h_offset,
    so the CM traces a circle of radius R_loop - h_offset.
    """
    return R_loop - h_offset
```

A.6.5 `physics/energy.py` (293 lines)

```python
"""
Analytical energy-balance solver for minimum release height.

Three models:
  1. Frictionless sliding block (reference case)
  2. Rolling sphere on flat surface (reference case)
  3. Rolling sphere on two rails with losses (prediction model)

Plus a waterfall breakdown function that shows how each physics correction
incrementally builds the prediction — directly generates content for the
memo appendix (30/100 points for equation development).

Dynamics principles:
  - Work-energy theorem (energy conservation from release to loop top)
  - Rotational dynamics (rolling without slipping, I = 2/5 mR^2)
  - Two-rail contact geometry (Bachman 1985)
  - Newton's 2nd law radial (centripetal condition at loop top)
"""
import numpy as np
from .constants import G, METERS_TO_INCHES, INCHES_TO_METERS, BallProperties
from .geometry import compute_contact_geometry, compute_effective_loop_radius
```

```python
def frictionless_block_height(R_loop: float) -> float:
    """
    Reference case 1: frictionless sliding block.

    Energy conservation: mgh = mg(2R) + (1/2)mv^2
    Critical condition at top: v^2 = gR (from N=0 in centripetal eq)
    Result: h_min = (5/2) * R_loop
    """
    return 2.5 * R_loop


def flat_rolling_height(R_loop: float) -> float:
    """
    Reference case 2: sphere rolling without slipping on flat surface.

    Energy conservation with rotation: mgh = mg(2R) + (7/10)mv^2
    where (7/10) comes from KE_trans + KE_rot = (1/2)mv^2 + (1/5)mv^2
    Critical condition at top: v^2 = gR
    Result: h_min = (27/10) * R_loop = 2.7 * R_loop
    """
    return 2.7 * R_loop


def two_rail_height(ball: BallProperties, R_loop: float, theta_deg: float,
                    C_rr: float = None, f_transition: float = 0.0,
                    safety_margin: float = 0.0) -> dict:
    """
    Prediction model: minimum release height for a sphere rolling on two rails
    to complete a vertical circular loop.

    Parameters:
        ball: BallProperties dataclass
        R_loop: loop radius in meters (rail centerline)
        theta_deg: ramp angle in degrees from horizontal
        C_rr: rolling resistance coefficient (overrides ball default if given)
        f_transition: fraction of KE lost at ramp-to-loop transition (0 to 1)
        safety_margin: additional height in meters added to prediction

    Returns dict with h_min_m, h_min_in, h_predicted_in, and energy breakdown.
    """
    if C_rr is None:
        C_rr = ball.C_rr

    theta = np.radians(theta_deg)
    geom = compute_contact_geometry(ball.radius_m)
    h_off = geom["h_offset"]
    r_eff = geom["r_eff"]
    KE_factor = geom["KE_factor"]
    R_c = compute_effective_loop_radius(R_loop, h_off)

    if R_c <= 0:
        raise ValueError(
            f"Effective loop radius R_c = {R_c*1000:.1f}mm is non-positive. "
            f"Loop radius {R_loop*1000:.1f}mm is too small for ball offset {h_off*1000:.1f}mm."
        )

    # Critical speed squared at loop top (Newton's 2nd law, radial, N=0)
    v_top_sq = G * R_c
```

```python
    # KE at loop top = (1/2) * m * v_top^2 * KE_factor
    KE_top = 0.5 * ball.mass_kg * v_top_sq * KE_factor

    # Rolling resistance in the loop
    # N_avg over a full loop ~ 3*m*g for a ball barely completing
    # (centripetal term averages to 2mg, gravity component averages to mg)
    # Path length = 2*pi*R_c
    W_loop_rr = C_rr * ball.mass_kg * G * 3.0 * (2 * np.pi * R_c)

    # Full energy balance from release to loop top:
    # m*g*[h + h_off*cos(theta)] = m*g*[2R_loop - h_off] + KE_top + W_ramp + W_loop + W_transition
    #
    # W_ramp = C_rr * m*g * h / tan(theta)              [linear in h]
    # W_transition ~ f_transition * m*g * [h - h_off*(1-cos(theta))]  [linear in h]
    #
    # Collecting h terms on LHS:
    # m*g*h * [1 - C_rr/tan(theta) - f_transition] = RHS (constants)

    rhs = (ball.mass_kg * G * (2 * R_loop - h_off * (1 + np.cos(theta)))
           + KE_top + W_loop_rr
           - f_transition * ball.mass_kg * G * h_off * (1 - np.cos(theta)))

    lhs_coefficient = ball.mass_kg * G * (1 - C_rr / np.tan(theta) - f_transition)

    if lhs_coefficient <= 0:
        h_min = float('inf')
    else:
        h_min = rhs / lhs_coefficient

    h_min_in = h_min * METERS_TO_INCHES
    h_predicted_in = np.ceil((h_min_in + safety_margin * METERS_TO_INCHES) * 10) / 10

    return {
        "h_min_m": h_min,
        "h_min_in": h_min_in,
        "h_predicted_in": h_predicted_in,
        "geom": geom,
        "R_c": R_c,
        "v_top_sq": v_top_sq,
        "KE_top": KE_top,
        "W_loop_rr": W_loop_rr,
        "KE_factor": KE_factor,
    }


def compute_all_predictions(R_loop_in: float, theta_deg: float,
                            balls: dict, C_rr_overrides: dict = None,
                            f_transition: float = 0.0,
                            safety_margin_in: float = 0.0) -> dict:
    """
    Compute predictions for all balls and both reference cases.

    Parameters:
        R_loop_in: loop radius in inches
        theta_deg: ramp angle in degrees
        balls: dict of BallProperties
        C_rr_overrides: dict {ball_name: C_rr_value}
        f_transition: transition loss fraction
        safety_margin_in: safety margin in inches
    """
    R_loop = R_loop_in * INCHES_TO_METERS
```

```python
    safety_m = safety_margin_in * INCHES_TO_METERS

    results = {
        "R_loop_in": R_loop_in,
        "theta_deg": theta_deg,
        "ref_block_in": frictionless_block_height(R_loop) * METERS_TO_INCHES,
        "ref_rolling_in": flat_rolling_height(R_loop) * METERS_TO_INCHES,
        "balls": {},
    }

    for name, ball in balls.items():
        C_rr = C_rr_overrides.get(name, ball.C_rr) if C_rr_overrides else ball.C_rr
        pred = two_rail_height(ball, R_loop, theta_deg, C_rr=C_rr,
                               f_transition=f_transition, safety_margin=safety_m)
        results["balls"][name] = pred

    return results


def compute_waterfall(ball: BallProperties, R_loop: float, theta_deg: float,
                      C_rr: float = None, f_transition: float = 0.0,
                      safety_margin_in: float = 0.0) -> list:
    """
    Compute the waterfall breakdown showing how each correction changes
    the predicted height. Returns a list of dicts, one per step.

    This directly generates content for the memo appendix (30/100 points
    for "clear presentation of equation development").

    Each step adds one physics correction on top of the previous result.
    """
    if C_rr is None:
        C_rr = ball.C_rr

    theta = np.radians(theta_deg)
    geom = compute_contact_geometry(ball.radius_m)
    h_off = geom["h_offset"]
    r_eff = geom["r_eff"]
    R_c = compute_effective_loop_radius(R_loop, h_off)
    KE_factor = geom["KE_factor"]
    v_top_sq = G * R_c

    steps = []

    # Step 1: Frictionless sliding block
    h1 = 2.5 * R_loop
    steps.append({
        "step": 1,
        "name": "Frictionless sliding block",
        "principle": "Energy conservation (no rotation, no friction)",
        "formula": "h = 5/2 * R_loop",
        "latex": r"T_1 + V_1 = T_2 + V_2 \quad\Rightarrow\quad \frac{1}{2}mv_{\text{top}}^2 +
         mg(2R) = mgh \quad\Rightarrow\quad h = \frac{5}{2}R",
        "h_m": h1,
        "h_in": h1 * METERS_TO_INCHES,
        "delta_in": 0.0,
        "source": "Fundamental - Work-Energy Theorem",
    })

    # Step 2: Add rotation (flat surface rolling)
    h2 = 2.7 * R_loop
```

```python
    steps.append({
        "step": 2,
        "name": "Add rotational KE (flat surface)",
        "principle": "Rolling without slipping, I = 2/5 mR^2, KE_rot = 1/5 mv^2",
        "formula": "h = 27/10 * R_loop",
        "latex": r"T = \frac{1}{2}mv_G^2 + \frac{1}{2}I_G\omega^2, \quad I_G = \frac{2}{5}mR^2,
        \quad \omega = \frac{v}{R} \quad\Rightarrow\quad T = \frac{7}{10}mv^2
        \quad\Rightarrow\quad h = \frac{27}{10}R",
        "h_m": h2,
        "h_in": h2 * METERS_TO_INCHES,
        "delta_in": (h2 - h1) * METERS_TO_INCHES,
        "source": "Rotational dynamics - rolling constraint v = wR",
    })

    # Step 3: Two-rail geometry correction (no losses yet)
    h3 = (2 * R_loop - h_off * (1 + np.cos(theta))) + R_c * KE_factor / 2
    steps.append({
        "step": 3,
        "name": "Two-rail geometry correction",
        "principle": (f"Effective rolling radius r_eff = {r_eff*1000:.2f} mm < R =
        {ball.radius_m*1000:.1f} mm. "
                      f"Ball spins faster, KE_factor = {KE_factor:.3f} (vs 1.4 on flat). "
                      f"CM path radius R_c = R_loop - h_offset = {R_c*1000:.1f} mm."),
        "formula": "r_eff = R * h_offset / (R + r_rail); KE_factor = 1 + 2/5 * (R/r_eff)^2",
        "latex": (
            r"r_{\text{eff}} = \frac{R \cdot h_{\text{offset}}}{R + r_{\text{rail}}}, \quad"
            r"\omega = \frac{v}{r_{\text{eff}}}, \quad"
            r"T = \frac{1}{2}mv^2\!\left[1 + \frac{2}{5}\!\left(\frac{R}
        {r_{\text{eff}}}\right)^{\!2}\right]"
        ),
        "h_m": h3,
        "h_in": h3 * METERS_TO_INCHES,
        "delta_in": (h3 - h2) * METERS_TO_INCHES,
        "source": "Bachman (1985), Am. J. Phys. 53(8), 765",
    })

    # Step 4: Add rolling resistance
    # Solve from scratch with rolling resistance (same structure as two_rail_height)
    W_loop_rr = C_rr * ball.mass_kg * G * 3.0 * (2 * np.pi * R_c)
    rhs4 = (ball.mass_kg * G * (2 * R_loop - h_off * (1 + np.cos(theta)))
            + 0.5 * ball.mass_kg * v_top_sq * KE_factor + W_loop_rr)
    lhs4 = ball.mass_kg * G * (1 - C_rr / np.tan(theta))
    h4 = rhs4 / lhs4 if lhs4 > 0 else float('inf')
    steps.append({
        "step": 4,
        "name": "Add rolling resistance",
        "principle": (f"C_rr = {C_rr:.3f}. Ramp loss = C_rr*mg*h/tan(theta). "
                      f"Loop loss = C_rr*mg*3*(2*pi*R_c) with N_avg ~ 3mg."),
        "formula": "W_rr = C_rr * N * ds (integrated over path)",
        "latex": r"U_{NC} = \int \mathbf{F} \cdot d\mathbf{r} = C_{rr} \int N\, ds, \quad
        W_{\text{ramp}} = \frac{C_{rr}\, mg\, h}{\tan\theta}, \quad W_{\text{loop}} = C_{rr}\cdot
        3mg \cdot 2\pi R_c",
        "h_m": h4,
        "h_in": h4 * METERS_TO_INCHES,
        "delta_in": (h4 - h3) * METERS_TO_INCHES,
        "source": "Rolling resistance (engineering literature)",
    })

    # Step 5: Add transition loss
    if f_transition > 0:
        rhs5 = (ball.mass_kg * G * (2 * R_loop - h_off * (1 + np.cos(theta)))
                + 0.5 * ball.mass_kg * v_top_sq * KE_factor + W_loop_rr
```

```
                - f_transition * ball.mass_kg * G * h_off * (1 - np.cos(theta)))
        lhs5 = ball.mass_kg * G * (1 - C_rr / np.tan(theta) - f_transition)
        h5 = rhs5 / lhs5 if lhs5 > 0 else float('inf')
    else:
        h5 = h4
    steps.append({
        "step": 5,
        "name": "Add transition loss",
        "principle": (f"f_transition = {f_transition:.0%} of KE lost at ramp-to-loop junction "
                      f"due to abrupt curvature change."),
        "formula": "W_trans = f_trans * KE_entry",
        "latex": r"W_{\text{trans}} = f_{\text{trans}} \cdot T_{\text{entry}} \quad \text{(Wang et
         al. 2021: jerk at curvature discontinuity)}",
        "h_m": h5,
        "h_in": h5 * METERS_TO_INCHES,
        "delta_in": (h5 - h4) * METERS_TO_INCHES,
        "source": "Wang et al. (2021), Am. J. Phys. 89(6), 583",
    })

    # Step 6: Safety margin
    h6_in = np.ceil((h5 * METERS_TO_INCHES + safety_margin_in) * 10) / 10
    steps.append({
        "step": 6,
        "name": "Safety margin",
        "principle": (f"+{safety_margin_in:.1f} in margin, rounded UP to 0.1 in. "
                      f"Accounts for trial-to-trial variability (3 successive trials required)."),
        "formula": "h_final = ceil((h_min + margin) * 10) / 10",
        "latex": r"h_{\text{final}} = \left\lceil (h_{\min} + \Delta h_{\text{margin}}) \times 10
         \right\rceil / 10",
        "h_m": h6_in * INCHES_TO_METERS,
        "h_in": h6_in,
        "delta_in": h6_in - h5 * METERS_TO_INCHES,
        "source": "Competition rules - 99-point penalty for failure",
    })

    return steps
```

A.6.6 `physics/ode_model.py` (199 lines)

```
"""
ODE-based simulation of a sphere rolling on a ramp + vertical circular loop.

Parameterizes the track as: ramp segment (arc length 0 to L_ramp) then
circular loop (arc length L_ramp to L_ramp + 2*pi*R_c).

Uses scipy.integrate.solve_ivp with terminal events:
  - Ball detaches from track (normal force N <= 0)
  - Ball stalls in the loop (v <= 0)
  - Ball completes the loop (s >= L_total)

Dynamics principles:
  - Newton's 2nd law (tangential: gravity + rolling resistance)
  - Newton's 2nd law (radial: centripetal condition for normal force)
  - Rolling constraint with effective radius from two-rail geometry
"""
import numpy as np
from scipy.integrate import solve_ivp
from .constants import G
```

```python
def build_track(R_loop: float, theta_deg: float, h_release: float,
                h_offset: float, R_c: float):
    """
    Build a track geometry function.

    Track consists of:
    1. Straight ramp from release point to loop bottom
    2. Circular loop of CM radius R_c

    Returns:
        track_func: callable(s) -> (x, y, phi, kappa, segment)
        L_ramp: arc length of ramp
        L_total: total arc length
    """
    theta = np.radians(theta_deg)

    # Ramp length from height
    L_ramp = h_release / np.sin(theta)
    L_loop = 2 * np.pi * R_c
    L_total = L_ramp + L_loop

    # Starting position (top of ramp, ball center coordinates)
    x_start = -L_ramp * np.cos(theta)
    y_start = h_release + h_offset * np.cos(theta)

    # Loop center (ball center at bottom is at (0, h_offset))
    loop_cx = 0.0
    loop_cy = h_offset + R_c

    def track_func(s):
        s = max(0, min(s, L_total))  # clamp

        if s <= L_ramp:
            # On the ramp
            x = x_start + s * np.cos(theta)
            y = y_start - s * np.sin(theta)
            phi = -theta  # going downhill
            kappa = 0.0
            return x, y, phi, kappa, "ramp"
        else:
            # In the loop
            s_loop = s - L_ramp
            phi_loop = s_loop / R_c  # 0 at bottom, pi at top, 2pi back

            x = loop_cx + R_c * np.sin(phi_loop)
            y = loop_cy - R_c * np.cos(phi_loop)

            kappa = 1.0 / R_c
            return x, y, phi_loop, kappa, "loop"

    return track_func, L_ramp, L_total


def simulate(ball_radius: float, ball_mass: float, R_loop: float,
             theta_deg: float, h_release: float, C_rr: float,
             KE_factor: float, h_offset: float) -> dict:
    """
    Simulate the ball rolling down the ramp and through the loop.

    Returns dict with t, s, v, x, y, N arrays plus completion status.
    """
```

```python
    R_c = R_loop - h_offset
    track, L_ramp, L_total = build_track(R_loop, theta_deg, h_release, h_offset, R_c)
    theta = np.radians(theta_deg)

    def ode(t, state):
        s, v = state
        if v <= 1e-10:
            v = 1e-10

        x, y, phi, kappa, segment = track(s)

        # Normal force
        if segment == "ramp":
            N = ball_mass * G * np.cos(theta)
        else:
            # N = mv^2/R_c + mg*cos(phi_loop)
            # Bottom (phi=0): N = mv^2/R_c + mg (correct)
            # Top (phi=pi): N = mv^2/R_c - mg (N=0 when v^2=gR_c)
            N = ball_mass * (v**2 / R_c + G * np.cos(phi))

        # Tangential acceleration
        if segment == "ramp":
            a_gravity = G * np.sin(theta)  # positive = downhill
        else:
            a_gravity = -G * np.sin(phi)  # negative when going up

        # Rolling resistance (opposes motion)
        a_rr = -C_rr * max(N, 0) / ball_mass * np.sign(v)

        # Divide by KE_factor for effective mass with rotation
        a = (a_gravity + a_rr) / KE_factor

        return [v, a]

    # Event: ball leaves track (N <= 0 in loop)
    def ball_leaves_track(t, state):
        s, v = state
        if s < L_ramp + 0.001:  # 1mm buffer past transition
            return 1.0
        x, y, phi, kappa, segment = track(s)
        N = ball_mass * (v**2 / R_c + G * np.cos(phi))
        return N

    ball_leaves_track.terminal = True
    ball_leaves_track.direction = -1

    # Event: ball stalls (v near 0) in loop
    def ball_stalls(t, state):
        s, v = state
        if s < L_ramp:
            return 1.0
        return v - 1e-8  # detect near-zero velocity (above the 1e-10 clamp)

    ball_stalls.terminal = True
    ball_stalls.direction = -1

    # Event: ball completes loop
    def loop_complete(t, state):
        s, v = state
        return s - L_total
```

```python
    loop_complete.terminal = True
    loop_complete.direction = 1

    # Estimate time and solve
    v_avg_est = max(np.sqrt(G * h_release), 0.1)
    t_max = max(3.0 * L_total / v_avg_est, 3.0)
    t_eval = np.linspace(0, t_max, 2000)

    sol = solve_ivp(ode, [0, t_max], [0.0, 1e-6], method='RK45',
                    t_eval=t_eval,
                    events=[ball_leaves_track, ball_stalls, loop_complete],
                    max_step=0.002, rtol=1e-6, atol=1e-8)

    # Extract results
    t_out = sol.t
    s_out = sol.y[0]
    v_out = sol.y[1]

    # Compute positions and normal forces
    positions = np.array([track(si)[:2] for si in s_out])
    x_out = positions[:, 0]
    y_out = positions[:, 1]

    N_out = np.zeros_like(t_out)
    for i, (si, vi) in enumerate(zip(s_out, v_out)):
        xi, yi, phi_i, kappa_i, seg_i = track(si)
        if seg_i == "ramp":
            N_out[i] = ball_mass * G * np.cos(theta)
        else:
            N_out[i] = ball_mass * (vi**2 / R_c + G * np.cos(phi_i))

    # Determine outcome (events: 0=leaves_track, 1=stalls, 2=loop_complete)
    completed = len(sol.t_events[2]) > 0
    failure_s = None
    failure_reason = None
    if len(sol.y_events[0]) > 0:
        failure_s = sol.y_events[0][0][0]
        failure_reason = "left_track"
    elif len(sol.y_events[1]) > 0:
        failure_s = sol.y_events[1][0][0]
        failure_reason = "stalled"

    return {
        "t": t_out,
        "s": s_out,
        "v": v_out,
        "x": x_out,
        "y": y_out,
        "N": N_out,
        "completed": completed,
        "failure_s": failure_s,
        "failure_reason": failure_reason,
        "L_ramp": L_ramp,
        "L_total": L_total,
        "track_func": track,
    }
```

A.6.7 `app.py` (510 lines)

```
"""
ME 2030 Loop-the-Loop Prediction Tool
=====================================
Predicts minimum release heights for spheres completing a vertical circular
loop on a two-rail track. Built for the ME 2030 Spring 2026 term project.

Dynamics principles used:
  - Work-energy theorem (energy conservation)
  - Rotational dynamics (rolling without slipping)
  - Two-rail contact geometry (Bachman 1985)
  - Newton's second law (centripetal condition at loop top)

Run locally:  streamlit run app.py
Deploy:       see render.yaml
"""
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import streamlit as st

from physics.constants import BALLS, G, RAIL_RADIUS, INCHES_TO_METERS, METERS_TO_INCHES
from physics.geometry import compute_contact_geometry
from physics.energy import (two_rail_height, compute_all_predictions, compute_waterfall)
from physics.ode_model import simulate

# ============================================================
# Page Config
# ============================================================
st.set_page_config(page_title="Loop Predictor", layout="wide")

# ============================================================
# Apple Liquid Glass CSS
# ============================================================
st.markdown("""
<style>
    @import url('https://fonts.googleapis.com/css2?
        family=Inter:wght@300;400;500;600;700&display=swap');

    html, body, [class*="st-"] {
        font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
    }

    .stApp {
        background: linear-gradient(135deg, #F5F5F7 0%, #E8E8ED 50%, #F5F5F7 100%);
    }

    section[data-testid="stSidebar"] {
        background: rgba(255, 255, 255, 0.72) !important;
        backdrop-filter: blur(20px) saturate(180%);
        -webkit-backdrop-filter: blur(20px) saturate(180%);
        border-right: 1px solid rgba(255, 255, 255, 0.3);
    }

    .stButton > button {
        background: linear-gradient(135deg, #0071E3 0%, #0077ED 100%);
        color: white;
        border: none;
```

```
        border-radius: 12px;
        padding: 0.6rem 1.4rem;
        font-weight: 500;
        transition: all 0.3s cubic-bezier(0.25, 0.1, 0.25, 1);
        box-shadow: 0 2px 8px rgba(0, 113, 227, 0.3);
    }

    .stButton > button:hover {
        transform: scale(1.02);
        box-shadow: 0 4px 16px rgba(0, 113, 227, 0.4);
    }

    div[data-testid="stMetric"] {
        background: rgba(255, 255, 255, 0.65);
        backdrop-filter: blur(12px);
        border-radius: 14px;
        border: 1px solid rgba(255, 255, 255, 0.4);
        padding: 1rem 1.2rem;
        box-shadow: 0 2px 16px rgba(0, 0, 0, 0.04);
    }

    h1, h2, h3 {
        color: #1D1D1F;
        font-weight: 600;
        letter-spacing: -0.02em;
    }

    h1 { font-size: 2.2rem; }

    #MainMenu {visibility: hidden;}
    header {visibility: hidden;}
    footer {visibility: hidden;}

    ::-webkit-scrollbar { width: 6px; }
    ::-webkit-scrollbar-track { background: transparent; }
    ::-webkit-scrollbar-thumb { background: rgba(0,0,0,0.15); border-radius: 3px; }
</style>
""", unsafe_allow_html=True)

# ============================================================
# Title
# ============================================================
st.title("Loop-the-Loop Predictor")
st.caption("ME 2030 Spring 2026 — Minimum release height prediction tool")

# ============================================================
# Sidebar: Inputs
# ============================================================
st.sidebar.header("Competition Inputs")
R_loop_in = st.sidebar.number_input("Loop Radius (inches)", min_value=1.0, max_value=20.0,
                                    value=5.0, step=0.1)
theta_deg = st.sidebar.slider("Ramp Angle (degrees)", min_value=5, max_value=80, value=30)

R_loop_interp = st.sidebar.selectbox("Loop Radius Measured To:",
    ["Inside surface of rails", "Rail centerline"],
    help="Based on project diagram, R appears measured to inside track surface")

# Apply interpretation correction
R_loop_effective_in = R_loop_in
if R_loop_interp == "Inside surface of rails":
    R_loop_effective_in = R_loop_in + RAIL_RADIUS * METERS_TO_INCHES
```

```python
st.sidebar.header("Safety & Losses")
safety_margin_in = st.sidebar.slider("Safety Margin (inches)", 0.0, 3.0, 0.5, 0.1)
f_transition = st.sidebar.slider("Transition Loss Fraction", 0.0, 0.25, 0.05, 0.01,
    help="Fraction of KE lost at ramp-to-loop junction (Wang et al. 2021)")

st.sidebar.header("Rolling Resistance (C_rr)")
C_rr_overrides = {}
for bname, ball in BALLS.items():
    C_rr_overrides[bname] = st.sidebar.slider(
        f"C_rr - {bname}", min_value=0.0, max_value=0.20,
        value=ball.C_rr, step=0.001, format="%.3f")

# ============================================================
# Compute predictions
# ============================================================
R_loop_m = R_loop_effective_in * INCHES_TO_METERS
results = compute_all_predictions(R_loop_effective_in, theta_deg, BALLS,
                                  C_rr_overrides, f_transition, safety_margin_in)

# Check for infinite predictions (low angle + high C_rr)
for bname, pred in results["balls"].items():
    if pred["h_min_m"] == float('inf'):
        st.warning(f"{bname}: No finite solution at this ramp angle and C_rr. Increase angle or
        decrease C_rr.")

# ============================================================
# Section 1: Competition Mode
# ============================================================
st.header("Predictions")

ball_names = list(BALLS.keys())
cols = st.columns(len(ball_names))
for col, bname in zip(cols, ball_names):
    pred = results["balls"][bname]
    col.metric(label=f"{bname}", value=f'{pred["h_predicted_in"]:.1f} in',
               delta=f'{pred["h_min_in"] - results["ref_rolling_in"]:+.2f} vs rolling')

total = sum(results["balls"][n]["h_predicted_in"] for n in ball_names)
st.info(f"Competition Score (if all succeed): **{total:.1f}**  |  "
        f"Ref Block: {results['ref_block_in']:.1f} in  |  Ref Rolling:
        {results['ref_rolling_in']:.1f} in")

# ============================================================
# Section 2: Waterfall Breakdown (CENTERPIECE)
# ============================================================
st.header("How We Got These Numbers")
st.caption("Each step applies one physics correction. This chain IS the equation development for
        the appendix.")

waterfall_ball = st.selectbox("Show waterfall for:", list(BALLS.keys()), key="wf_ball")
ball_obj = BALLS[waterfall_ball]
steps = compute_waterfall(ball_obj, R_loop_m, theta_deg,
                          C_rr=C_rr_overrides[waterfall_ball],
                          f_transition=f_transition,
                          safety_margin_in=safety_margin_in)

# Waterfall table
wf_rows = []
for s in steps:
    wf_rows.append({
        "Step": s["step"],
```

```
        "Correction": s["name"],
        "h (in)": f'{s["h_in"]:.2f}',
        "Delta": f'{s["delta_in"]:+.2f}' if s["step"] > 1 else "base",
        "Principle": s["principle"][:90] + ("..." if len(s["principle"]) > 90 else ""),
        "Source": s["source"],
    })
st.dataframe(pd.DataFrame(wf_rows), use_container_width=True, hide_index=True)

# Waterfall cascade chart
fig_wf, ax_wf = plt.subplots(figsize=(10, 4))
fig_wf.patch.set_facecolor('#F5F5F7')
ax_wf.set_facecolor('#F5F5F7')

labels = [s["name"] for s in steps]
values = [s["h_in"] for s in steps]
deltas = [s["delta_in"] for s in steps]

colors = ["#0071E3", "#34C759", "#FF9500", "#FF3B30", "#AF52DE", "#8E8E93"]
bottoms = [0] + values[:-1]
for i, (label, val, delta, bottom) in enumerate(zip(labels, values, deltas, bottoms)):
    if i == 0:
        ax_wf.bar(i, val, color=colors[i % len(colors)], alpha=0.85, width=0.6)
    else:
        ax_wf.plot([i - 0.7, i - 0.3], [bottom, bottom], color='#6E6E73', lw=0.8, ls='--')
        ax_wf.bar(i, delta, bottom=bottom, color=colors[i % len(colors)], alpha=0.85, width=0.6)
    ax_wf.text(i, val + 0.1, f'{val:.2f}"', ha='center', fontsize=8, fontweight=500,
        color='#1D1D1F')

ax_wf.set_xticks(range(len(labels)))
ax_wf.set_xticklabels([l.replace(" ", "\n") for l in labels], fontsize=7)
ax_wf.set_ylabel("Height (inches)")
ax_wf.set_title(f"Prediction Waterfall — {waterfall_ball} Ball", fontweight=600)
ax_wf.grid(axis='y', alpha=0.2)
ax_wf.spines['top'].set_visible(False)
ax_wf.spines['right'].set_visible(False)
fig_wf.tight_layout()
st.pyplot(fig_wf)
plt.close(fig_wf)

# Expandable derivation details
with st.expander("Full derivation details (for appendix)", expanded=False):
    for s in steps:
        st.markdown(f"### Step {s['step']}: {s['name']}")
        st.latex(s.get("latex", ""))
        st.caption(f"{s['principle']}")
        st.markdown(f"**Result:** {s['h_in']:.3f} in  ({s['delta_in']:+.3f} in)  —  *
        {s['source']}*")
        st.divider()

# ============================================================
# Section 2b: Dynamics Principles Reference (class notation)
# ============================================================
st.header("Dynamics Principles Applied")
st.caption("Equations from ME 2030 Dynamics Vocabulary mapped to this model")

princ_col1, princ_col2 = st.columns(2)

with princ_col1:
    st.markdown("**Work-Energy Theorem**")
    st.latex(r"T_1 + V_1 + U_{NC} = T_2 + V_2")
    st.caption("Energy conservation with non-conservative work (rolling resistance)")
```

```
    st.markdown("**Kinetic Energy (Rolling Rigid Body)**")
    st.latex(r"T = \frac{1}{2}mv_G^2 + \frac{1}{2}I_G\omega^2")
    st.caption("Total KE = translational + rotational")

    st.markdown("**Moment of Inertia (Solid Sphere)**")
    st.latex(r"I_G = \frac{2}{5}mR^2")
    st.caption("Used with rolling constraint: omega = v / r_eff")

with princ_col2:
    st.markdown("**Newton's 2nd Law (Radial, Loop Top)**")
    st.latex(r"\sum F_n = ma_n \quad\Rightarrow\quad N + mg = \frac{mv^2}{R_c}")
    st.caption("Setting N=0 gives critical speed: v² = gR_c")

    st.markdown("**Normal-Tangential Acceleration**")
    st.latex(r"a_n = \frac{v^2}{\rho}, \qquad a_t = \dot{v}")
    st.caption("Path coordinates — rho = R_c in the loop")

    st.markdown("**Work of Non-Conservative Forces**")
    st.latex(r"U_{NC} = \int \mathbf{F} \cdot d\mathbf{r} = C_{rr}\!\int N\, ds")
    st.caption("Rolling resistance dissipates energy along the track path")

# ============================================================
# Section 3: Two-Rail Geometry
# ============================================================
st.header("Two-Rail Geometry")
geom_rows = []
for bname, ball in BALLS.items():
    geom = compute_contact_geometry(ball.radius_m)
    geom_rows.append({
        "Ball": bname,
        "R (mm)": f"{ball.radius_m*1000:.2f}",
        "r_eff (mm)": f"{geom['r_eff']*1000:.2f}",
        "h_offset (mm)": f"{geom['h_offset']*1000:.2f}",
        "R/r_eff": f"{ball.radius_m/geom['r_eff']:.3f}",
        "KE_factor": f"{geom['KE_factor']:.3f}",
        "KE vs flat": f"+{(geom['KE_factor']/1.4 - 1)*100:.0f}%",
    })
st.dataframe(pd.DataFrame(geom_rows), use_container_width=True, hide_index=True)

# ============================================================
# Section 4: Track Diagram
# ============================================================
st.header("Track Geometry")
fig_track, ax_track = plt.subplots(figsize=(10, 6))
fig_track.patch.set_facecolor('#F5F5F7')
ax_track.set_facecolor('#F5F5F7')

sample_geom = compute_contact_geometry(list(BALLS.values())[0].radius_m)
R_c_sample = R_loop_m - sample_geom["h_offset"]
theta_rad = np.radians(theta_deg)
h_pred_m = results["balls"]["Steel"]["h_min_m"]
L_ramp = h_pred_m / np.sin(theta_rad)

# Draw ramp
ramp_x = np.array([-L_ramp * np.cos(theta_rad), 0]) * 1000
ramp_y = np.array([h_pred_m + sample_geom["h_offset"] * np.cos(theta_rad),
                   sample_geom["h_offset"]]) * 1000
ax_track.plot(ramp_x, ramp_y, color='#1D1D1F', lw=2.5)

# Draw loop
```

```
phi_arr = np.linspace(0, 2 * np.pi, 200)
loop_cx = 0.0
loop_cy = (sample_geom["h_offset"] + R_c_sample) * 1000
loop_x = loop_cx + R_c_sample * 1000 * np.sin(phi_arr)
loop_y = loop_cy - R_c_sample * 1000 * np.cos(phi_arr)
ax_track.plot(loop_x, loop_y, color='#1D1D1F', lw=2.5)

# Ball at start
ax_track.plot(ramp_x[0], ramp_y[0], 'o', color='#0071E3', markersize=12, zorder=5)

# Annotations
ax_track.annotate(f'R = {R_loop_effective_in:.1f} in', xy=(loop_cx, loop_cy),
                  fontsize=10, ha='center', color='#0071E3', fontweight=500)
ax_track.annotate(f'h = {results["balls"]["Steel"]["h_predicted_in"]:.1f} in',
                  xy=(ramp_x[0], ramp_y[0] + 8), fontsize=9, color='#FF3B30',
                  ha='center', fontweight=500)
ax_track.set_aspect('equal')
ax_track.set_xlabel('x (mm)')
ax_track.set_ylabel('y (mm)')
ax_track.grid(True, alpha=0.15)
ax_track.spines['top'].set_visible(False)
ax_track.spines['right'].set_visible(False)
fig_track.tight_layout()
st.pyplot(fig_track)
plt.close(fig_track)

# ============================================================
# Section 5: Energy Breakdown
# ============================================================
st.header("Energy Breakdown")
fig_energy, ax_energy = plt.subplots(figsize=(10, 5))
fig_energy.patch.set_facecolor('#F5F5F7')
ax_energy.set_facecolor('#F5F5F7')
for i, bname in enumerate(ball_names):
    pred = results["balls"][bname]
    ball_geom = compute_contact_geometry(BALLS[bname].radius_m)
    m = BALLS[bname].mass_kg
    v_top_sq = pred["v_top_sq"]
    KE_factor_val = pred["KE_factor"]

    total_E = m * G * pred["h_min_m"]
    PE_top = m * G * (2 * R_loop_m - 2 * ball_geom["h_offset"])
    KE_trans = 0.5 * m * v_top_sq
    KE_rot = 0.5 * m * v_top_sq * (KE_factor_val - 1)

    bottom_val = 0
    for label, val, color in [
        ("PE at top", PE_top, "#0071E3"),
        ("KE translational", KE_trans, "#34C759"),
        ("KE rotational", KE_rot, "#FF3B30"),
        ("Losses", max(total_E - PE_top - KE_trans - KE_rot, 0), "#8E8E93"),
    ]:
        pct = val / total_E * 100 if total_E > 0 else 0
        ax_energy.bar(i, pct, bottom=bottom_val, width=0.6,
                      label=label if i == 0 else "", color=color, alpha=0.85)
        bottom_val += pct

ax_energy.set_xticks(range(len(ball_names)))
ax_energy.set_xticklabels(ball_names)
ax_energy.set_ylabel("% of Release PE")
ax_energy.legend(loc='upper right')
```

```python
ax_energy.spines['top'].set_visible(False)
ax_energy.spines['right'].set_visible(False)
fig_energy.tight_layout()
st.pyplot(fig_energy)
plt.close(fig_energy)

# ============================================================
# Section 6: Simulator
# ============================================================
st.header("Simulation")
sim_col1, sim_col2 = st.columns([1, 1])
with sim_col1:
    sim_ball = st.selectbox("Ball", list(BALLS.keys()), key="sim_ball")
with sim_col2:
    sim_h_in = st.number_input("Release height (in)", min_value=1.0, max_value=50.0,
                               value=float(results["balls"][sim_ball]["h_predicted_in"]),
                               step=0.1)

if st.button("Run Simulation", type="primary"):
    ball = BALLS[sim_ball]
    geom = compute_contact_geometry(ball.radius_m)
    h_release_m = sim_h_in * INCHES_TO_METERS

    with st.spinner("Solving ODE..."):
        sim_result = simulate(ball.radius_m, ball.mass_kg, R_loop_m, theta_deg,
                              h_release_m, C_rr_overrides[sim_ball],
                              geom["KE_factor"], geom["h_offset"])

    if len(sim_result["v"]) == 0:
        st.error("Simulation returned no data. Check inputs.")
    elif sim_result["completed"]:
        st.success(f"Loop completed. Final speed: {sim_result['v'][-1]:.2f} m/s")
    else:
        st.error(f"Loop NOT completed. Reason: {sim_result.get('failure_reason', 'unknown')}")

    if len(sim_result["v"]) > 0:
        # Velocity + Normal Force plot
        fig_sim, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 5), sharex=True)
        fig_sim.patch.set_facecolor('#F5F5F7')
        ax1.set_facecolor('#F5F5F7')
        ax2.set_facecolor('#F5F5F7')

        s_mm = sim_result["s"] * 1000
        ax1.plot(s_mm, sim_result["v"], color=ball.color, lw=1.5)
        ax1.axvline(x=sim_result["L_ramp"] * 1000, color='#8E8E93', ls='--', lw=0.8,
                    label='Loop entry')
        ax1.set_ylabel("Speed (m/s)")
        ax1.text(0.98, 0.95, r'$T = \frac{1}{2}mv^2 \cdot \mathrm{KE}_{\mathrm{factor}}$',
                 transform=ax1.transAxes, ha='right', va='top', fontsize=9,
                 color='#6E6E73', style='italic')
        ax1.legend(fontsize=8)

        ax2.plot(s_mm, sim_result["N"] * 1000, color=ball.color, lw=1.5)
        ax2.axhline(y=0, color='#FF3B30', ls='--', lw=0.8, alpha=0.6)
        ax2.axvline(x=sim_result["L_ramp"] * 1000, color='#8E8E93', ls='--', lw=0.8)
        ax2.set_ylabel("Normal Force (mN)")
        ax2.set_xlabel("Arc length (mm)")
        ax2.text(0.98, 0.95, r'$N = \frac{mv^2}{R_c} + mg\cos\phi$',
                 transform=ax2.transAxes, ha='right', va='top', fontsize=9,
                 color='#6E6E73', style='italic')
```

```
        for ax in [ax1, ax2]:
            ax.spines['top'].set_visible(False)
            ax.spines['right'].set_visible(False)

        fig_sim.tight_layout()
        st.pyplot(fig_sim)
        plt.close(fig_sim)

        # Animation
        st.subheader("Animation")
        placeholder = st.empty()
        track_fn = sim_result["track_func"]
        s_track = np.linspace(0, sim_result["L_total"], 500)
        track_pts = np.array([track_fn(si)[:2] for si in s_track])

        skip = max(1, len(sim_result["t"]) // 40)  # ~40 frames for Render compat
        for idx in range(0, len(sim_result["t"]), skip):
            fig_a, ax_a = plt.subplots(figsize=(8, 6))
            fig_a.patch.set_facecolor('#F5F5F7')
            ax_a.set_facecolor('#F5F5F7')
            ax_a.plot(track_pts[:, 0] * 1000, track_pts[:, 1] * 1000,
                      color='#1D1D1F', lw=2)
            ax_a.plot(sim_result["x"][idx] * 1000, sim_result["y"][idx] * 1000, 'o',
                      color=ball.color, markersize=14, zorder=5)
            ax_a.set_aspect('equal')
            ax_a.set_xlabel('x (mm)')
            ax_a.set_ylabel('y (mm)')
            ax_a.set_title(
                f't={sim_result["t"][idx]:.3f}s   v={sim_result["v"][idx]:.2f} m/s   '
                f'N={sim_result["N"][idx]*1000:.1f} mN', fontsize=10)
            ax_a.grid(True, alpha=0.15)
            ax_a.spines['top'].set_visible(False)
            ax_a.spines['right'].set_visible(False)
            placeholder.pyplot(fig_a)
            plt.close(fig_a)
            time.sleep(0.03)

# ============================================================
# Section 7: Memo Comparison Table (R=5in, theta=20 and 50)
# ============================================================
st.header("Memo Reference Tables")
st.caption("Required for memo section II: R_loop=5 in, theta = 20 and 50 degrees")

memo_R = 5.0
memo_rows = []
for theta_val in [20, 50]:
    mr = compute_all_predictions(memo_R, theta_val, BALLS, C_rr_overrides, f_transition, 0)
    bh = mr["ref_block_in"]
    rh = mr["ref_rolling_in"]
    row = {"Angle": f"{theta_val} deg", "Block": f"{bh:.2f}", "Rolling": f"{rh:.2f}"}
    for bname in ball_names:
        h = mr["balls"][bname]["h_min_in"]
        row[bname] = f"{h:.2f}"
        row[f"{bname} vs Roll"] = f"{h - rh:+.2f}"
    memo_rows.append(row)

st.dataframe(pd.DataFrame(memo_rows), use_container_width=True, hide_index=True)

# ============================================================
# Section 8: Sensitivity Analysis
# ============================================================
```

```python
st.header("Sensitivity Analysis")
st.caption("Parameter uncertainty impact on predictions")

sens_rows = []
for bname in ball_names:
    base = results["balls"][bname]["h_min_in"]
    crr_base = C_rr_overrides[bname]

    p_lo = two_rail_height(BALLS[bname], R_loop_m, theta_deg,
                           C_rr=crr_base * 0.5, f_transition=f_transition)
    p_hi = two_rail_height(BALLS[bname], R_loop_m, theta_deg,
                           C_rr=crr_base * 1.5, f_transition=f_transition)

    # Alternative R_loop interpretation
    if R_loop_interp == "Rail centerline":
        R_alt = (R_loop_in + RAIL_RADIUS * METERS_TO_INCHES) * INCHES_TO_METERS
    else:
        R_alt = R_loop_in * INCHES_TO_METERS
    p_alt = two_rail_height(BALLS[bname], R_alt, theta_deg,
                            C_rr=crr_base, f_transition=f_transition)

    sens_rows.append({
        "Ball": bname,
        "Base (in)": f"{base:.2f}",
        "C_rr −50%": f"{p_lo['h_min_in']:.2f} ({p_lo['h_min_in']-base:+.2f})",
        "C_rr +50%": f"{p_hi['h_min_in']:.2f} ({p_hi['h_min_in']-base:+.2f})",
        "Alt R interp": f"{p_alt['h_min_in']:.2f} ({p_alt['h_min_in']-base:+.2f})",
    })

st.dataframe(pd.DataFrame(sens_rows), use_container_width=True, hide_index=True)
```

A.7 References

1. **Bachman, R. A.** (1985). "Sphere rolling down a grooved track." *American Journal of Physics*, 53(8), 765. — Foundational paper deriving the effective rolling radius for a sphere on two parallel rails.
2. **Hibbeler, R. C.** (2021). *Engineering Mechanics: Dynamics*, 15th Ed., Pearson. — Course textbook. Notation conventions (T, V, U_{NC}, I_G, I_{IC}) and the IC method for rolling-body kinematics follow this text.
3. **West, B., Barrett, R., & Vender, B.** (2024). "Deep in Galileo's Groove." *The Physics Teacher*, 62(2), 104. — Recent extension of Bachman's groove dynamics, validating the r_{eff} correction at varied groove geometries.
4. **Wang, Y., et al.** (2021). "Energy loss and jerk on the loop-the-loop." *American Journal of Physics*, 89(6), 583. — Documents the energy lost at the curvature discontinuity between ramp and loop ("jerk" loss).
5. **Bertran, O.** (2020). "A revised solution for a sphere rolling in a vertical loop." UPC repository preprint. — Analyses the slip condition near the loop top where $N \to 0$ and friction simultaneously vanishes.